ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ МОЛДОВЫ

Факультет информатики, вычислительной техники
и микроэлектроники

ПРОГРАММИРОВАНИЕ МОБИЛЬНЫХ ПРИЛОЖЕНИЙ (PAM)

ОТЧЁТ

Лабораторная работа № 2

Тема: Статический UI приложения — экраны темы в Material 3

Проект: Go2Point — планирование поездок на мероприятия

Номер темы по каталогу: _____

Выполнил: студент группы TI-236

Todorov Veaceslav

Проверил: унив. лектор Ionel Antohi

Кишинэу — 2026

1. Анализ предметной области и цель работы

1.1. Назначение приложения Go2Point

Go2Point представляет собой приложение для подготовки поездки на концерт, выставку, фестиваль или другое мероприятие. Пользователь задаёт город отправления, направление, сведения о событии, даты пребывания и бюджет. Интерфейс предусматривает просмотр составного маршрута, сравнение альтернатив по стоимости и времени, сохранённые поездки, редактирование плана и личный профиль с отзывами.

На этапе L2 разработан визуальный прототип: прикладные данные заданы непосредственно в исходных файлах, а подключение к сервисам поиска билетов, размещения и аутентификации отсутствует. Такой формат позволяет рассмотреть состав экранов, удобство размещения информации и согласованность оформления до реализации серверной части.

1.2. Цель и используемые средства

Цель работы — построить экраны приложения средствами Flutter, применить единую визуальную тему Material 3 и выделить повторяющиеся элементы в переиспользуемые компоненты. В работе используется язык Dart; исходный код редактируется в IDE, а версии проекта сохраняются в Git и публикуются в GitHub. Библиотека google_fonts используется для шрифта Quicksand, а flutter_svg подключена для SVG-ресурсов.

Основные средства построения интерфейса — Scaffold, AppBar, SafeArea, Column, Row, Expanded, SingleChildScrollView, PageView, Container, TextField и кнопки Material. Длинные формы помещены в прокручиваемые области, а повторяющиеся блоки оформлены карточками с одинаковыми отступами и скруглениями.

2. Выполнение заданий лабораторной работы

2.1. Задание 1 — структура проекта и работа с Git

Требование L2: организовать проект с отдельными файлами экранов, каталогом переиспользуемых виджетов и последовательной историей изменений. В Go2Point роль каталога lib/screens/ из методических указаний выполняет lib/pages/: каждый из восьми экранов находится в собственном Dart-файле. Настройки оформления дополнительно выделены в lib/theme/.

```
lib/
├── main.dart
├── pages/
│   ├── login_page.dart
│   ├── register_page.dart
│   ├── trip_planner_page.dart
│   ├── route_details_page.dart
│   ├── alternative_routes_page.dart
│   ├── saved_routes_page.dart
│   ├── edit_route_page.dart
│   └── profile_page.dart
├── theme/
│   ├── app_colors.dart
│   └── app_theme.dart
└── widgets/              # общие поля, кнопки и карточки
```

Файл main.dart содержит точку входа, MaterialApp и выбор стартового экрана. Каталог pages объединяет интерфейсы страниц, widgets — компоненты, которые повторяются в формах и маршрутах, theme — генерацию палитры и настройку ThemeData. Ресурсы приложения располагаются в assets, а проверки палитры — в test/app_colors_test.dart. Отдельный каталог data в текущей версии не создан; демонстрационные данные находятся в файлах соответствующих экранов.

Листинг 1 - Подключение общей темы и стартовой страницы. Источник: lib/main.dart, строки 15–22.

```
return MaterialApp(
  debugShowCheckedModeBanner: false,

  title: 'Go2Point',

  theme: AppTheme.light(),

  home: const TripPlannerPage(),
```

Ниже приведена полная история git log --oneline на момент подготовки отчёта. Изменения экранов объединены в общие коммиты; отдельного коммита для каждого экрана, как предусмотрено минимальным требованием L2, в этой истории нет.

```
c5693c6 All tasks lab2 done
5bc7e46 All Pages UI Done
0a46018 Main Page Design
859221b Merge remote main
1d5c47c Initial commit
3560500 Initial Flutter project
```

2.2. Задание 2 — визуальная тема Material 3

Требование L2: определить собственный seed-цвет, сформировать ColorScheme.fromSeed и получать цвета интерфейса через Theme.of(context).colorScheme. В приложении сохранён светлый лавандовый характер первоначального дизайна. Основной цвет #B8A1FF выбран для акцентных кнопок и элементов маршрута, а светлые поверхности позволяют разделять насыщенные информацией блоки поездки.

В lib/theme/app_colors.dart метод AppColors.light создаёт светлую ColorScheme с вариантом fidelity. Производные оттенки берутся из сгенерированной схемы; для светлых акцентов используются её контейнерные тона. Поэтому изменение seedColor и hot reload меняют акценты, подложки и оттенки текста из одного места, а не требуют редактирования каждой страницы.

Листинг 2 - Исходный цвет и генерация палитры. Источник: lib/theme/app_colors.dart, строки 5–12.

```
static const seedColor = Color(0xFFB8A1FF);

static ColorScheme light({Color seed = seedColor}) {
  final generated = ColorScheme.fromSeed(
    seedColor: seed,
    brightness: Brightness.light,
    dynamicSchemeVariant: DynamicSchemeVariant.fidelity,
  );
```

Роли в схеме разделены по назначению: primary — главный акцент, onPrimary — текст поверх него, surface — фон, onSurface — основной текст, onSurfaceVariant — второстепенные подписи, surfaceContainerHighest — фон полей. Цвета success и warning вынесены в AppStatusColors, расширение ThemeExtension: зелёный и жёлтый продолжают обозначать статусы независимо от выбранного брендового оттенка.

Листинг 3 - Тема Material 3 и единая типографика. Источник: lib/theme/app_theme.dart, строки 10–17.

```
return ThemeData(
  useMaterial3: true,
  colorScheme: scheme,
  scaffoldBackgroundColor: scheme.surface,
  textTheme: GoogleFonts.quicksandTextTheme().apply(
    bodyColor: scheme.onSurface,
    displayColor: scheme.onSurface,
  ),
```

AppTheme.light также задаёт оформление AppBar и InputDecorationTheme и подключает AppStatusColors. На страницах и в общих виджетах схема получается из контекста; например, в AppButton фон определяется scheme.primary, а цвет надписи — scheme.onPrimary. Это сохраняет согласованную пару фона и текста при смене темы.

2.3. Задание 3 — экраны и демонстрационные данные

Требование L2: построить статические экраны по предметной области, использовать реалистичные данные и базовые компоненты Material. Подготовлены восемь страниц: вход, регистрация, планировщик поездки, детали маршрута, альтернативные маршруты, сохранённые маршруты, редактирование маршрута и профиль. Подробное описание каждой страницы и места для изображений приведены в разделе 3.

В качестве примеров используются поездки из Кишинёва в Будапешт, Бухарест, Вену и Прагу, концерт Metallica и другие мероприятия. Карточки содержат города, даты, цену в евро, продолжительность поездки, цепочку транспорта и сведения об отеле. В деталях маршрута последовательно показаны перелёт, трансфер, проживание, билет на событие, местный транспорт и возвращение в аэропорт.

Листинг 4 - Реалистичные данные одного этапа маршрута. Источник: lib/pages/route_details_page.dart, строки 94–99.

```
const RouteStepCard(
  icon: Icons.flight_takeoff_rounded,
  title: 'Flight',
  subtitle: 'Chisinau → Budapest',
  price: '€145',
),
```

Альтернативы представлены горизонтальным PageView с тремя вариантами: экономичным, сбалансированным и наиболее комфортным. Сохранённые поездки показаны вертикальным PageView с четырьмя карточками, а профиль содержит два отзыва. Это фактические объёмы текущего демонстрационного набора; требование методических

указаний о не менее шести элементах в каждом списке пока не выполнено для этих коллекций.

Для предварительного просмотра карты используется цветной Container с иконкой карты и поясняющей надписью; интерактивная карта не подключена. Аватар профиля также представлен иконкой. Таким образом, основные визуальные блоки доступны без загрузки прикладных изображений из сети.

В планировщике и редакторе уже добавлены локальные действия: выбор дат, изменение допустимого отклонения бюджета, пересчёт стоимости и сброс правок. Для этих действий, а также индикаторов PageView, применяются StatefulWidget и setState. Они выходят за минимальную статическую часть L2; серверное сохранение и управление общим состоянием приложения не реализованы.

2.4. Задание 4 — статические формы входа и регистрации

Требование L2: показать поля и кнопку входа без валидации и логики аутентификации. LoginPage содержит логотип, приветственный текст, поле e-mail, скрытое поле пароля, кнопку Sign In и ссылку Sign Up. RegisterPage дополняет форму именем и подтверждением пароля. Во всех случаях используется общий компонент AppTextField, построенный на TextField.

Листинг 5 - Поле пароля и кнопка входа без обработчика авторизации. Источник: lib/pages/login_page.dart, строки 65–74.

```
const AppTextField(
  label: 'Password',
  hint: 'Enter your password',
  obscureText: true,
),

const SizedBox(height: 32),

// Sign In button
AppButton(text: 'Sign In', onPressed: () {}),
```

Передача obscureText: true скрывает пароль при вводе. Form, TextFormField и validator для этих форм не используются; нажатия кнопок пока не проверяют данные, не создают учётную запись и не обращаются к backend. Основная кнопка обёрнута в AppButton и реализована через ElevatedButton, тогда как в примере задания указан FilledButton.

2.5. Задание 5 — переиспользуемые виджеты

Требование L2: выделить не менее двух параметризованных виджетов и использовать их повторно. В проекте общие элементы вынесены в lib/widgets/. Текст, значки, значения и обработчики передаются через конструкторы, поэтому изменение внешнего вида компонента применяется ко всем местам его использования.

Компонент и файл в lib/widgets/	**Назначение и использование**
AppTextField app_text_field.dart	Общее поле ввода: вход, регистрация, планировщик и редактор маршрута; используется также внутри редактируемых карточек.
AppButton app_button.dart	Основная кнопка: вход, регистрация, планировщик, детали и редактирование маршрута.

Компонент и файл в lib/widgets/	Назначение и использование
RouteSummaryCard route_summary_card.dart	Город, событие и период поездки: детали маршрута и сравнение альтернатив.
RouteStepCard route_step_card.dart	Один этап поездки; повторяется шесть раз на странице деталей.
AlternativeRouteCard / SavedRouteCard	Параметризованные карточки сравнения и сохранённых поездок на соответствующих страницах.
EditableRouteStepCard / EditableEventCard	Редактируемые блоки транспорта и мероприятия на странице изменения маршрута.
UserReviewCard user_review_card.dart	Отзыв с рейтингом, датой и действиями; повторяется в профиле.

Листинг 6 - Параметры общей карточки поездки. Источник: lib/widgets/route_summary_card.dart, строки 5–14.

```
final String destination;
final String event;
final String dateRange;

const RouteSummaryCard({
  super.key,
  required this.destination,
  required this.event,
  required this.dateRange,
});
```

RouteSummaryCard принимает направление, название мероприятия и период поездки. Одинаковая карточка используется в деталях и на странице альтернатив, благодаря чему основные сведения о поездке имеют одно оформление. Аналогичный подход применён к кнопкам и полям: страницы задают содержимое, а компонент определяет отступы, форму и работу с общей темой.

Листинг 7 - Получение цветов темы в общей кнопке. Источник: lib/widgets/app_button.dart, строки 18–27.

```
final scheme = Theme.of(context).colorScheme;
return SizedBox(
  width: double.infinity,
  height: 58,
  child: ElevatedButton(
    onPressed: onPressed,

    style: ElevatedButton.styleFrom(
      backgroundColor: scheme.primary,
      foregroundColor: scheme.onPrimary,
```

Дополнительно используются SectionTitle для нумерованных разделов формы, PriceSummaryCard для итоговой стоимости, EditedRouteBadge для отметки изменённого маршрута и AppLogo для брендового блока. Компоненты остаются отдельными классами, а не копируются целиком между страницами.

2.6. Задание 6 — доступ к экранам и навигация

Требование L2: обеспечить возможность открыть каждый экран во время защиты. Методические указания предлагают стартовую страницу с кнопками и Navigator.push либо

NavigationBar для основных разделов. Это временная демонстрационная навигация, а маршрутизация с go_router отнесена к L3.

В текущей версии стартовой страницей является TripPlannerPage, что видно в листинге 1. Обработчики переходов на других экранах пока пустые: Navigator.push, таблица маршрутов и NavigationBar в проекте не добавлены. Поэтому доступ ко всем страницам через интерфейс пока не реализован; их можно просмотреть по отдельности, временно меняя стартовый экран в коде, но это не заменяет требуемую навигацию на защите.

Отдельная тестовая шторка не создавалась: дальнейшая работа предусматривает собственную навигацию приложения. Этот пункт описывает текущее состояние проекта и не заявляется как завершённая система переходов.

3. Представление результатов — экраны приложения

В этом разделе предусмотрено по одному месту для каждого из восьми скриншотов. Изображения следует получить на эмуляторе или физическом устройстве и вставить перед соответствующей подписью; до вставки изображений наличие этих подписей само по себе не подтверждает запуск приложения на устройстве.

3.1. Вход (Sign In)

Страница показывает форму входа в Go2Point: приветствие, e-mail, пароль, основную кнопку и ссылку на регистрацию. Основные элементы — Scaffold, AppBar, SafeArea, SingleChildScrollView, Column, AppTextField, AppButton и TextButton; содержимое формы не связано с серверной авторизацией.

Исходный файл: lib/pages/login_page.dart.

Рисунок 1 - Скриншот страницы "Вход"

3.2. Регистрация (Sign Up)

Страница содержит поля имени, e-mail, пароля и подтверждения пароля, кнопку регистрации и ссылку на вход. Компоновка повторяет стиль экрана входа и использует те же AppTextField и AppButton; ввод пароля скрыт, проверки совпадения и сохранения аккаунта нет.

Исходный файл: lib/pages/register_page.dart.

Рисунок 2 - Скриншот страницы "Регистрация"

3.3. Планировщик поездки (Plan my trip)

Планировщик разделён на блоки направления, мероприятия, пребывания и бюджета. Используются SectionTitle, AppTextField, Row с Expanded, заглушка карты и кнопка построения; локально доступны выбор дат и расчёт диапазона бюджета, а генерация настоящего маршрута пока отсутствует.

Исходный файл: lib/pages/trip_planner_page.dart.

Рисунок 3 - Скриншот страницы "Планировщик поездки"

3.4. Детали маршрута (Your route)

Страница показывает сводку поездки, шесть последовательных этапов и итоговую стоимость. RouteSummaryCard, повторяющиеся RouteStepCard и PriceSummaryCard составляют прокручиваемый маршрут; кнопки сохранения и перехода к другим действиям представлены визуально.

Исходный файл: lib/pages/route_details_page.dart.

Рисунок 4 - Скриншот страницы "Детали маршрута"

3.5. Альтернативные маршруты (Alternative routes)

Экран позволяет визуально сравнить три демонстрационных варианта: экономичный, сбалансированный и комфортный. Горизонтальный PageView содержит AlternativeRouteCard с ценой, временем, транспортом, отелем и отклонением от бюджета; индикатор отмечает текущую карточку.

Исходный файл: lib/pages/alternative_routes_page.dart.

Рисунок 5 - Скриншот страницы "Альтернативные маршруты"

3.6. Сохранённые маршруты (Saved routes)

Экран представляет четыре сохранённые поездки в Будапешт, Бухарест, Вену и Прагу. Вертикальный PageView и SavedRouteCard показывают даты, стоимость, время и транспорт; кнопки просмотра, редактирования и удаления пока не меняют хранилище данных.

Исходный файл: lib/pages/saved_routes_page.dart.

Рисунок 6 - Скриншот страницы "Сохранённые маршруты"

3.7. Редактирование маршрута (Edit route)

Редактор группирует поля перелёта, трансферов, отеля, мероприятия и местного транспорта. Используются EditableRouteStepCard, EditableEventCard, AppTextField и EditedRouteBadge; правки локально отражаются в сравнении исходной и новой цены и могут быть сброшены, но не сохраняются на сервер.

Исходный файл: lib/pages/edit_route_page.dart.

Рисунок 7 - Скриншот страницы "Редактирование маршрута"

3.8. Профиль (Profile)

Профиль содержит аватар-заглушку, имя, e-mail, статистику сохранённых поездок и отзывов, а также две карточки UserReviewCard. Экран построен из прокручиваемой Column, Row, иконок, текстовых блоков и кнопок; рейтинг представлен звёздами, а сведения о пользователе заданы в коде.

Исходный файл: lib/pages/profile_page.dart.

Рисунок 8 - Скриншот страницы "Профиль"

3.9. Проверка реализации и границы текущей версии

Для темы выполнен анализ lib с результатом No issues found. В test/app_colors_test.dart находятся две проверки: смена seed меняет акценты, контейнеры и оттенки текста; контраст пар primary/onPrimary и primaryContainer/onPrimaryContainer превышает 4,5 для выбранного фиолетового и дополнительного зелёного seed. Оба теста палитры прошли при проверке реализации темы; это не является проверкой всех сценариев приложения или подтверждением отсутствия переполнений на любом размере экрана.

Проверка общего тестового набора отдельно не заявляется: test/widget_test.dart сохраняет шаблонный тест счётчика со ссылкой на MyApp, тогда как корневой класс проекта называется Go2PointApp. Кроме того, AppLogo ссылается на assets/icons/go2point_icon.svg, которого нет среди текущих файлов assets; этот ресурс необходимо добавить либо исправить путь до проверки форм входа и регистрации на устройстве.

Пункт	**Фактическое состояние**
Экраны	В исходном коде присутствуют восемь страниц. Сопоставление с номером темы каталога требует отдельной спецификации Go2Point.
Тема и общие компоненты	Подключены ColorScheme.fromSeed, ThemeData и параметризованные виджеты; значения цветов вынесены из экранов в тему.
Данные и состояние	Прикладные данные демонстрационные; коллекции содержат 3 альтернативы, 4 сохранённых маршрута и 2 отзыва. Имеется локальное состояние форм.
Навигация и демонстрация	Переходы между страницами ещё не подключены. Скриншоты и проверка на устройстве дополняются отдельно.
История разработки	История Git приведена полностью; изменения нескольких экранов объединены в общие коммиты.

Таким образом, отчёт фиксирует реализованный интерфейс и явно отделяет его от функций следующих этапов. Перед окончательным экспортом для сдачи необходимо вставить изображения, указать номер утверждённой темы и привести демонстрацию экранов к требованиям защиты.

4. Вывод

В ходе лабораторной работы был разработан визуальный прототип приложения Go2Point для планирования поездок на мероприятия. В проекте подготовлены восемь экранов, охватывающих вход и регистрацию, подготовку поездки, работу с маршрутами и просмотр профиля. Оформление централизовано с помощью ThemeData и ColorScheme.fromSeed, благодаря чему основные оттенки интерфейса изменяются через исходный цвет. Повторяющиеся поля, кнопки и карточки вынесены в параметризованные виджеты, что упрощает поддержку единого стиля. Реалистичные демонстрационные данные и локальные действия в формах позволяют показать состав поездки и изменение её стоимости без подключения backend. Полученная основа готова к дальнейшему развитию, при этом навигация, расширение демонстрационных коллекций и окончательная проверка всех экранов на устройстве остаются следующими шагами.

5. Использование средств ИИ

При подготовке отчёта использовался ChatGPT (Codex) для анализа структуры проекта, подбора небольших фрагментов исходного кода и оформления материала по предоставленному образцу. Описания реализации сопоставлены с файлами текущего проекта; изображения экранов не генерировались. Итоговую проверку содержания и вставку скриншотов выполняет автор отчёта.

6. Библиография и использованные материалы

1. Antohi Ionel. Лабораторная работа № 2 — Статический UI приложения: экраны темы в Material 3. Методические указания по дисциплине PAM. Файл: Documents_univer/L2_UI_static_al_aplicatiei_RU.pdf.

2. Образец оформления отчёта по другой дисциплине. Файл: Documents_univer/Raport_laborator2.docx. Использован как образец структуры: титульный лист, анализ, проектирование, результаты, выводы и источники.

3. Исходный код Go2Point: lib/main.dart, lib/pages/, lib/theme/, lib/widgets/ и test/app_colors_test.dart. Версия интерфейса и темы: коммит c5693c6 «All tasks lab2 done».

7. Ссылки

Репозиторий проекта: https://github.com/SlavaMold/go2point

Зафиксированная версия L2:
https://github.com/SlavaMold/go2point/commit/c5693c65674fda3c4dd1818f8a64b087250fb96a